COMPILER CONSTRUCTION LAB · PA1 · GROUP 5

Run & Demo Runbook

From a fresh laptop to a running live demo: install, open, build, test, demo, and what to do when something breaks.

Tested on macOS with OpenJDK 27, Maven 3.9.15, Python 3.14. On Windows, replace `:` with `;` in every `-cp`.

1. What you need

Tool	Version	macOS	Windows	Linux (Debian/Ubuntu)
JDK	8 or newer	`brew install openjdk`	`winget install EclipseAdoptium.Temurin.21.JDK`	`sudo apt install openjdk-21-jdk`
Maven	3.3.9 or newer	`brew install maven`	`winget install Apache.Maven` (or unzip it and add `bin` to PATH)	`sudo apt install maven`
Python 3	any 3.x	`brew install python`	`winget install Python.Python.3.12`	usually preinstalled
IDE (optional)		IntelliJ IDEA (Community is enough) or VS Code with the "Extension Pack for Java"		

Check: `java -version`, `mvn -v`, `python3 --version` (on Windows `python --version`). Python is only needed for our test tools and the demo, not for building or grading.

2. Folder layout

```
PA1/                                (= the GitHub repository root)
├── README.md                       <- start here: overview, status, how to review
├── chocopy-pa1/                    <- the Maven project (this is what we submit)
│   ├── pom.xml                     (provided, do not edit)
│   ├── chocopy-ref.jar             (reference implementation)
│   ├── README.md                   (our write-up is at the end)
│   ├── src/main/jflex/chocopy/pa1/ChocoPy.jflex   <- OUR LEXER
│   ├── src/main/cup/chocopy/pa1/ChocoPy.cup       <- OUR GRAMMAR
│   ├── src/test/data/pa1/sample/                  (33 provided tests)
│   ├── src/test/data/pa1/edge_cases/              (our 44 tests)
│   ├── src/test/data/pa1/student_contributed/     (good.py, bad.py: graded)
│   └── tools/                      (difftest.py, fuzz.py, recovery_quality.py, ast_tree.py)
├── demo/                           (demo.sh + 5 demo programs)
├── docs/                           (these PDFs; HTML sources in docs/src/)
├── presentation/                   (slides .pptx + .pdf, Uni Passau / chair format)
├── scripts/                        (make_submission.sh, render_docs.sh)
└── submission/pa1_group_5.zip      (ready to upload)
```

3. Open the project in an IDE

IntelliJ IDEA

1. *File* → *Open…* → select `PA1/chocopy-pa1/pom.xml` → *Open as Project* → *Trust Project*.
2. Wait for the Maven import. Then *Maven* tool window → *Lifecycle* → double-click **package**. This generates the lexer and parser.
3. Generated sources appear under `target/generated-sources/`. **Never edit them**: edit `.jflex` / `.cup` and rebuild.
4. For the demo: *View* → *Appearance* → *Presentation Mode*, or set the editor font to ≥ 18 pt.

VS Code

1. *File* → *Open Folder…* → `PA1/chocopy-pa1`. Install "Extension Pack for Java" when it's suggested.
2. Open a terminal (*Terminal* → *New Terminal*) and use the commands below.

4. Build

```
cd PA1/chocopy-pa1
mvn clean package
```

What happens: JFlex turns `ChocoPy.jflex` into `ChocoPyLexer.java`. CUP turns `ChocoPy.cup` into `ChocoPyParser.java` + `ChocoPyTokens.java` and prints any grammar conflicts (we have none). javac compiles, and the result is `target/assignment.jar`. The only expected warnings: *Terminal "RESERVED"/"UNRECOGNIZED" was declared but never used*, which is intentional.

THE FIRST BUILD NEEDS THE INTERNET

`pom.xml` uses an open version range for Jackson, so Maven contacts Maven Central. If you get `502 Bad Gateway` or a timeout, simply run it again. Once everything is in `~/.m2`, you can build offline with `mvn -o clean package`. **Build on the presentation laptop the day before.**

5. Run it

Goal	Command (from `PA1/chocopy-pa1`)
Run all provided tests	`java -cp "chocopy-ref.jar:target/assignment.jar" chocopy.ChocoPy --pass=s --dir src/test/data/pa1/sample --test` Expected last line: `Tests: 33 passed, 0 failed, and 0 ignored.`
Run our edge cases	same, with `--dir src/test/data/pa1/edge_cases` → `44 passed`
Parse one file, print the JSON AST	`java -cp "chocopy-ref.jar:target/assignment.jar" chocopy.ChocoPy --pass=s FILE.py`
Same file with the reference parser	replace `--pass=s` with `--pass=r`
Write the JSON to a file	add `--out ast.json`
Show the AST as a readable tree	`... --pass=s --out ast.json FILE.py` then `python3 tools/ast_tree.py ast.json`
Show our token stream (INDENT/DEDENT)	`java -cp "chocopy-ref.jar:target/assignment.jar" chocopy.pa1.ChocoPyLexer FILE.py`
Reference token stream	replace `chocopy.pa1` with `chocopy.reference`
Parser trace (shifts, reductions, recovery)	add `--debug` to the parse command
Compare with the reference (grader's rule)	`python3 tools/difftest.py DIR_OR_FILE …`
Generate random test programs	`python3 tools/fuzz.py /tmp/fuzz 200 42` (count, seed) → then `difftest.py /tmp/fuzz`
Measure error recovery	`python3 tools/recovery_quality.py /tmp/fuzz 200` (takes a few minutes)

If there are syntax errors, the driver prints them as `file:line:col: Parse error near token …` instead of JSON. Use `--out` to get the JSON with its `errors` list anyway.

6. The live demo

Everything is scripted in `PA1/demo/demo.sh`. It shows each command before running it and waits for **Enter** (type `s` + Enter to skip a step).

```
cd PA1
bash demo/demo.sh
```

Before the talk

- ☐ `cd PA1/chocopy-pa1 && mvn clean package` the evening before, on the presenting laptop (needs the internet once).
- ☐ Do a full dry run of `demo.sh` that morning.
- ☐ Terminal font ≥ 20 pt (macOS Terminal: ⌘ +), light theme for the projector, window maximised, "Do Not Disturb" on.
- ☐ Have the slide PDF open as a fallback. The slides contain the real outputs below.

Script: what you run, what you see, what you say

#	Expected output	Say
1	`Tests: 33 passed, 0 failed` `Tests: 44 passed, 0 failed`	"All provided samples, plus our own 44 edge cases."

2	`line:2 col:1 --  --INDENT--`
	`line:4 col:1 --  --INDENT--`
	`line:5 col:1 --  --INDENT--`
	`line:6 col:1 ----DEDENT--`
	`line:6 col:1 ----DEDENT--`
	`line:8 col:1 ----DEDENT--`
		"Line 6 closes the `if` and the `for` at once, so it needs two DEDENTs. Each one comes from re-scanning the same whitespace after a `yypushback`."

3	`IfStmt [1, 1, 7, 1]`
	`  elseBody: IfStmt [3, 1, 7, 1]`
	`  elseBody: []`
		"`elif` becomes a nested IfStmt in the else-body. `else: pass` is an empty list, yet the statement still ends at the else-block's DEDENT (7,1). That rule tripped us up."

4	`IfExpr [1, 5, 1, 44]`
	`  condition: BinaryExpr or`
	`  left: UnaryExpr not`
	`  thenExpr: BinaryExpr +`
	`  left: BinaryExpr *`
	`  left: UnaryExpr -`
		"Precedence: unary minus binds tighter than `*`, `not` tighter than `or`, and the conditional expression is the loosest."

5	`5 files, 5 match the reference, 0 differ`	"Our differential tester uses the grader's own comparison rule."
6	Ours: `1:11 PLUS`, `4:13 COMMA`, `11:1 RESERVED` (3 errors) Reference: 8 errors (2:2, 4:1, 4:16, 7:1, 7:11, 10:1 …)	"Three real mistakes. We report three errors, and the reference reports eight. The cascade comes from resuming in the wrong mode."
6c	`ours statements on lines [10, 11, 12]` `reference statements on lines [1, 5, 8, 11, 12]`	"Line 10, `z = 5`, is correct code. The reference loses it, because the error on the next line is detected before the statement is reduced. Our default reductions keep it."
7	Reference: `4:1 UNRECOGNIZED` and `6:1 EOF` Ours: only `4:1`	"A bug we found: after a bad dedent, the reference's INDENTs and DEDENTs no longer balance, which produces a bogus EOF error."
8	`100 files, 100 match the reference, 0 differ`	"Fresh random programs with a new seed, right now."

TIMING

The full demo takes about 3 minutes. If the talk is already at time, show steps **2, 6 and 7** only (about 1 minute). Steps 1 and 8 can run while you talk.

Fallback plan

- **Build fails at the venue** (no network): use `mvn -o clean package`. If `target/` already exists, don't rebuild at all; the demo only needs `target/assignment.jar`.
- **Laptop/projector problem:** present from the PDF. The slides contain the same outputs.
- **A live question** ("what happens if…?"): write the snippet to a file and run `--pass=s` next to `--pass=r`. That's the most convincing answer there is.

7. Troubleshooting

Symptom	Cause → fix
`Could not find or load main class chocopy.ChocoPy`	Wrong folder or no build yet → `cd PA1/chocopy-pa1`, run `mvn clean package`, and check that `target/assignment.jar` exists.
Same error on Windows	Use `;` instead of `:` in `-cp "chocopy-ref.jar;target/assignment.jar"`.
`502 Bad Gateway` / `Could not resolve dependencies`	Maven Central was unreachable → retry, or `mvn -o …` once the dependencies are cached.
`WARNING: Final field … mutated reflectively`	JDK 24+ with Jackson in the provided code. Harmless; ignore it.
Tests fail right after editing `.jflex` / `.cup`	You forgot to rebuild. Always `mvn clean package` after editing, especially after adding a terminal.
CUP reports a shift/reduce or reduce/reduce conflict	The build stops. Read the conflict report: it names the state and both items. Usually a precedence is missing or two `error` rules overlap.
`python3: command not found` (Windows)	Use `python`, and run the commands from `demo.sh` by hand in Git Bash or WSL.
Output is human-readable errors, not JSON	Expected when the program has syntax errors. Add `--out file.json` to get the JSON.
difftest is slow	Each file starts two JVMs. It already runs in parallel, so use a smaller corpus for the demo.

8. Making the submission

The prepared file is `PA1/submission/pa1_group_5.zip`. To recreate it (and test it from a clean unzip) in one step:

```
cd PA1
bash scripts/make_submission.sh
```

1. The script already unzips it into an empty folder, builds it there and runs the sample tests; check that it ends with `OK`.
2. Upload `pa1_group_5.zip` to **SubmissionsDir** on Stud.IP **before Wed 04.11.2026, 23:59**. One member uploads; the latest upload counts.
3. Download it again from Stud.IP and repeat step 1. (Yes, really.)